

BANKING FINANCIAL STATEMENT REVIEW AUTOMATION

An AI-Assisted Audit Copilot — Advanced MVP Blueprint

Prepared for: Cognizant Technoverse Hackathon — Banking Track

Team Size: 7 Members | Build & Submission Window: 2 Days

August 2026

1. Executive Summary

This blueprint upgrades the original 7-member task plan into a hackathon-ready, judge-ready product: an “Audit Copilot” that ingests a bank's financial statements (PDF/Excel), runs deterministic mathematical and consistency checks, uses an LLM strictly for language, explanation and narrative work (never for arithmetic), links every finding to its source evidence, and auto-drafts a WP-514 working paper for human sign-off.

The core design principle is separation of certainty from judgment: anything that has one correct numeric answer (totals, cross-casting, prior-year deltas) is computed deterministically in code; anything that requires language or explanatory judgement (grammar, “why did this move” narratives, reviewer commentary) is delegated to an LLM that is only allowed to reason over already-validated numbers — never invent them.

The rest of this document gives a concrete technology stack, a layered architecture, full module-level design, a worked real-life example (“GreenPeak Bank Ltd.”) showing the pipeline catching four planted errors end-to-end, a revised 7-person task split with an hour-by-hour build timeline, and guidance on mapping the build to Cognizant's judging criteria (Impact, Innovation, Technical Strength, Scalability).

2. Problem Statement & Business Impact

Financial statement review in banking is high-stakes and still largely manual: reviewers cross-foot totals by hand, eyeball prior-year comparatives for “unusual movements,” manually trace note disclosures back to the primary statements, and write review commentary from scratch. This is slow, inconsistent across reviewers, and easy to get wrong under time pressure — exactly the kind of structured-but-tedious work that benefits from automation with a human still in control of judgement calls and sign-off.

2.1 Pain Points → What the Copilot Fixes

Manual Pain Point	Automated Fix
Re-adding totals/subtotals by hand across BS, P&L, Cash Flow	Deterministic math engine cross-foots every statement in seconds
Comparing current vs prior year figures line by line	Automated PY delta engine flags material % / absolute movements
Manually tracing a note figure back to the statement	Consistency engine matches note ↔ statement ↔ cash flow automatically
Writing review commentary and explaining anomalies	AI layer drafts reviewer-friendly, evidence-grounded narrative
Populating the WP-514 working paper by hand	Validated findings auto-populate WP-514 draft fields
No single view of status across a bank's statement set	Dashboard shows overall status, severity and drill-down evidence

3. Solution Overview

Positioning: this is not “AI that audits a bank.” It is a copilot that removes the mechanical 80% of review work (casting, comparison, tracing, drafting) so a human reviewer spends their time on judgement — materiality, root cause, and sign-off. Three design rules govern every module:

- Determinism for numbers. Every arithmetic or comparison check (totals, cross-casting, YoY %, note-to-statement matches) is plain code — Python/pandas — never an LLM call. This makes results reproducible and defensible.
- Grounding for language. The AI layer only ever receives already-validated figures and explicitly labelled findings; it is prompted to explain and phrase, not to calculate or invent numbers.
- Evidence for trust. Every finding — pass or fail — carries a pointer back to its source: document, page, table/note number, and row/cell reference, so a reviewer can verify in one click.

4. Advanced System Architecture

The MVP architecture from the original plan is preserved (extraction → canonical data → evidence mapping → parallel engines → findings → WP-514 → dashboard) and extended with an orchestration layer, a findings/risk aggregator, and clearly separated storage. Synthetic data generation stays fully outside the production pipeline, feeding it only as an input file, exactly like a real uploaded statement would.



Persistence: a single PostgreSQL (or SQLite for the hackathon) database holds jobs, canonical data, findings and WP-514 drafts; raw uploaded files sit in local/S3-style object storage keyed by job id. No message queue is

required at MVP scale — the orchestrator can run the three engines synchronously or as async tasks within one FastAPI process.

5. Recommended Tech Stack

Chosen for maximum library support for financial-document parsing and numeric work, while keeping the whole thing buildable by 7 people inside a 2-day window. A JS-only alternative is included for teams that want one language end-to-end.

Layer	Primary Choice	Why
Frontend / Dashboard	React + TypeScript, Vite, Tailwind CSS, Recharts	Fast to scaffold; Recharts gives quick status/severity charts; Tailwind speeds up styling under time pressure
PDF evidence viewer	react-pdf / pdf.js	Renders the exact page/row a finding points to, inline in the drill-down panel
Backend / Orchestrator	Python 3.11 + FastAPI	Async-friendly, auto-generated OpenAPI docs for inter-module contracts, plays well with data libraries
PDF extraction	pdfplumber + camelot-py (+ Tesseract OCR fallback)	Best open-source combo for text + ruled/unruled table extraction from statements
Excel extraction	openpyxl + pandas	Reads formulas/values, multiple sheets (BS, P&L, Notes) cleanly
Math / Consistency engines	Python + pandas + numpy (pure rule functions)	Deterministic, testable, no ML — exactly what auditable arithmetic needs
AI layer	Anthropic Claude API (Claude Sonnet 5), structured JSON output	Strong grounded-explanation and language-review quality; force JSON schema so output maps straight into findings
Grammar assist (deterministic backstop)	LanguageTool (open-source) as a first pass, LLM refines tone	Keeps spelling/grammar checks fast and free; LLM only polishes phrasing and explains anomalies
Database	PostgreSQL (Docker) — SQLite acceptable for demo-only	Relational fit for accounts x periods x findings; SQLite is a zero-setup fallback
File / evidence storage	Local disk or MinIO (S3-compatible)	Keeps raw files + page images addressable by job id
Auth	Skip or single shared demo login	Explicitly out of scope per hackathon guardrails — do not spend hours here
Deployment (demo)	Docker Compose, single host	One `docker compose up` for judges; no cloud infra needed for MVP

5.1 All-JavaScript Alternative (if the team prefers one language)

- Backend: Node.js + Express or Fastify, with pdf-parse / pdf.js for text and a table-extraction library (or a Python microservice just for camelot, called via HTTP, if table extraction quality matters more than stack purity).
- Math/consistency engines: plain TypeScript functions operating on the canonical JSON — same determinism rule applies regardless of language.
- Everything else (DB, AI layer, frontend) is unchanged.

6. Detailed Module Design

6.1 Document Extraction

Goal: turn an uploaded PDF/XLSX into raw text + tables + a first-pass label of which statement or note each table belongs to (Balance Sheet, P&L, Cash Flow, Equity Statement, Notes 1..n).

- PDF: pdfplumber for text and simple tables; camelot (lattice mode) for ruled tables typical of bank statements; Tesseract OCR fallback if a page has no extractable text layer (scanned document).
- Excel: openpyxl/pandas reads each sheet; sheet-name and header heuristics (“Total Assets”, “Net Profit”, “Cash and cash equivalents”) assign a statement label.
- Every extracted number keeps a pointer: {document_id, page, table_id, row_label, column/period}. This pointer is what the Evidence layer stores.
- Hackathon scope: optimize first for the team's own synthetic PDFs/XLSX (fixed layout), not arbitrary real-world formats — per the original guardrail.

6.2 Canonical Financial Data Model

All downstream engines read one normalized JSON shape, regardless of source format. Example (abridged):

```
{
  "bank_id": "GREENPEAK",
  "periods": ["FY2025", "FY2024"],
  "statements": {
    "balance_sheet": {
      "total_assets": {"FY2025": 1245000, "FY2024": 1102000, "evidence": {...}},
      "total_liabilities": {"FY2025": 1080000, "FY2024": 958000, "evidence": {...}},
      "total_equity": {"FY2025": 161000, "FY2024": 144000, "evidence": {...}},
      "cash_and_equivalents": {"FY2025": 125000, "FY2024": 98000, "evidence": {...}}
    },
    "profit_and_loss": { "net_income": {"FY2025": 21600, "FY2024": 17800, "evidence": {...}} },
    "cash_flow": { "closing_cash": {"FY2025": 120500, "FY2024": 98000, "evidence": {...}} },
    "notes": { "note_12_cash": {"FY2025": 120500, "FY2024": 98000, "evidence": {...}} }
  }
}
```

6.3 Mathematical Engine (Deterministic)

Pure functions, one per rule, each returning a pass/fail plus the exact numeric difference and the evidence pointers of every value it used. No LLM involvement.

Rule	Formula / Check
Balance sheet balances	Total Assets = Total Liabilities + Total Equity
P&L cross-cast	Total Income – Total Expenses = Net Profit/(Loss)
Cash flow cross-cast	Opening Cash + Net Cash Flow (Op+Inv+Fin) = Closing Cash
Equity roll-forward	Opening Equity + Net Income + Other Movements – Dividends = Closing Equity
Subtotal/footing	Every disclosed subtotal equals the sum of its component lines

6.4 Consistency + Prior-Year Engine (Deterministic)

- Cross-statement matches: Balance Sheet cash ↔ Cash Flow closing cash ↔ Note on cash & equivalents (must be equal, or flagged).
- Selected linkages: net income (P&L) feeding the equity roll-forward and the cash-flow statement's operating section starting point; dividends/tax/depreciation appearing consistently across statement and note.

- Prior-year comparison: for every mapped line item, compute absolute and percentage movement $FY(t)$ vs $FY(t-1)$; flag anything beyond a configurable materiality threshold (e.g. $>20\%$ and $>₹5$ Cr) as “unusual movement” for the AI layer to explain.

6.5 AI Layer (LLM-Assisted, Grounded)

The AI layer never sees raw documents directly for numeric work — it receives the canonical data plus the deterministic engines' findings, and is instructed to explain, phrase and summarize only. Three sub-tasks:

- Language review: spelling/grammar pass on extracted narrative text (director's report, note descriptions) — LanguageTool for a fast deterministic pass, LLM to suggest natural rephrasing for anything flagged.
- Anomaly explanation: given a flagged prior-year movement (e.g. “Other Income up 340%”) plus any related note text, produce a plausible, evidence-grounded explanation candidate and clearly mark it as a suggested explanation for reviewer confirmation, not a fact.
- Reviewer summary: a short, plain-English roll-up of overall findings by severity, written for a WP-514 reviewer, strictly built from the findings list it was given.

Prompting rule: system prompt instructs the model to only use figures present in the supplied JSON, to never introduce a number that isn't in context, and to output structured JSON (finding_id, explanation, confidence, caveats) so it drops straight into the findings table.

6.6 WP-514 Module

Once findings are validated, the module maps them into the agreed/prototype WP-514 template: reviewed item, expected value, actual value, variance, status (Pass/Exception), evidence reference, and an editable reviewer comment/sign-off field that stays human-controlled by default, matching the original guardrail.

6.7 Dashboard

- Upload screen with job status (Extracting → Reviewing → Done).
- Overall status header: Pass / Exceptions count, by severity (High/Medium/Low).
- Findings table, filterable by module (Math / Consistency / PY / AI / Disclosure), each row expandable to show the evidence (source page image + highlighted cell).
- WP-514 tab: the generated working paper, editable, exportable to PDF/Excel.

7. Real-Life Walkthrough: GreenPeak Bank Ltd.

A fictional synthetic case (Member 2's dataset, Case 9 style — “multiple issues in one bank”) showing the full pipeline. All figures are illustrative and in ₹ Crore.

Step 1 — Upload

The auditor uploads GreenPeak_FY2025_Statements.pdf, containing the Balance Sheet, P&L, Cash Flow Statement, Equity Statement and Notes 1–18, with FY2025 and FY2024 comparatives.

Step 2 — Extraction → Canonical Data

The extraction module identifies five statements and eighteen notes, and produces canonical JSON with evidence pointers for every cell, e.g. Total Assets FY2025 → {page: 2, table: “Balance Sheet”, row: “Total Assets”}.

Step 3 — Deterministic Checks Run

Check	Result	Evidence
Balance Sheet balances	FAIL — Total Assets ₹12,450 Cr ≠ Total Liabilities + Equity ₹12,410 Cr (diff ₹40 Cr)	Page 2, rows: Total Assets / Total Liabilities / Total Equity
Cash cross-statement match	FAIL — Balance Sheet cash ₹1,250 Cr vs Note 12 cash ₹1,205 Cr (diff ₹45 Cr)	Page 2 row: Cash & Equivalents; Page 9, Note 12
Prior-year movement — Other Income	FLAGGED — up 340% YoY (₹42 Cr → ₹185 Cr)	Page 4, P&L, row: Other Income
P&L cross-cast	PASS — Income – Expenses = Net Profit, ties out exactly	Page 4

Step 4 — AI Layer (Grounded)

Language review flags “recievables” in Note 7 and suggests “receivables.” For the Other Income spike, the AI layer is given only the validated figures and Note 15 text (“Gain on sale of investment property”) and produces a grounded, clearly-labelled candidate explanation: the movement is likely driven by a one-off gain disclosed in Note 15, and recommends the reviewer confirm this is non-recurring for forecasting purposes. It does not assert this as fact — it is flagged as a suggested explanation pending reviewer confirmation.

Step 5 — WP-514 Draft (excerpt)

Item Reviewed	Expected	Actual	Status	Comment
Balance Sheet equation	Assets = Liab + Equity	Diff ₹40 Cr	Exception	Reviewer to trace ₹40 Cr difference; math engine could not locate a matching entry
Cash — BS vs Note 12	Equal	Diff ₹45 Cr	Exception	Possible timing item or note error; needs reviewer confirmation
Other Income YoY movement	Within threshold	+340%	Exception (Explained)	AI-suggested cause: one-off gain per Note 15 — pending sign-off
P&L cross-cast	Ties out	Ties out	Pass	—

Step 6 — Dashboard

The dashboard shows GreenPeak's overall status as “3 Exceptions, 1 Pass,” with severity badges, and lets the reviewer click any exception to see the exact PDF page and highlighted row that produced it — closing the loop from raw document to actionable, evidence-backed finding, in seconds instead of hours.

8. Inter-Module Data Contracts (Sample)

A single findings schema keeps Math, Consistency/PY and AI outputs interchangeable for the aggregator and dashboard:

```
{
  "finding_id": "F-014",
  "module": "math_engine",
  "check": "balance_sheet_equation",
  "status": "exception",
  "severity": "high",
  "expected": 1241000,
  "actual": 1245000,
  "difference": 4000,
```

```

"evidence": [
  {"doc_id": "greenpeak_fy25.pdf", "page": 2, "row": "Total Assets"},
  {"doc_id": "greenpeak_fy25.pdf", "page": 2, "row": "Total Liabilities"},
  {"doc_id": "greenpeak_fy25.pdf", "page": 2, "row": "Total Equity"}
],
"ai_explanation": null
}

```

9. Revised 7-Member Task Distribution & Tech Ownership

#	Member	Workstream	Owns (Tech)
1	Domain + WP-514 Lead	Defines checks, banking ratios, validates WP-514 fields with mentor	WP-514 field mapping logic, requirements doc
2	Dummy Dataset	5–10 fictional bank cases, 2-year statements, seeded errors, ground truth	Synthetic PDF/XLSX generator scripts, ground-truth JSON
3	Extraction	PDF/Excel ingestion, table/text extraction, canonical JSON output	pdfplumber, camelot, openpyxl, evidence pointers
4	Math Engine	Deterministic BS/P&L/CF/Equity checks	Python + pandas rule functions, unit tests
5	Consistency + PY	Cross-statement checks, YoY movement analysis	Python rule functions on canonical JSON
6	AI Layer	Grammar, anomaly explanation, reviewer summary	Claude API integration, prompt templates, JSON-schema output
7	Dashboard + Integration	Upload UI, findings view, WP-514 view, end-to-end demo	React/TS frontend, FastAPI orchestrator, Docker Compose

9.1 Suggested 2-Day (48-Hour) Timeline

A 2-day window is enough to build a genuinely working MVP rather than a rushed demo — use the extra time for real integration testing and a rehearsed narrative, not more features.

When	Milestone
Day 1, Hr 0–2	Kickoff: lock canonical JSON schema, evidence-pointer format & WP-514 field list — every member agrees before writing code (Members 1, 3, 4, 5, 6, 7 together)
Day 1, Hr 2–6	Member 2 delivers first 2–3 synthetic cases (clean + 1 error case); Member 3 gets basic PDF/XLSX extraction working against them; Member 7 scaffolds repo, dashboard shell and upload flow
Day 1, Hr 6–12	Member 3 completes extraction across all planned formats; Members 4 & 5 build and unit-test rule engines against canonical JSON stubs (don't wait on real extraction — use fixture JSON)
Day 1, Hr 12–18	Member 2 finishes remaining synthetic cases + ground truth; Member 6 wires the AI layer to real findings with grounded prompts; Member 7 builds the findings table + evidence drill-down UI
Day 1, Hr 18–24	First full integration pass: extraction → engines → findings, running end-to-end on at least 1 synthetic case; log every integration bug for Day 2
Day 2, Hr 0–6	Fix Day 1 integration bugs; Member 7 finishes WP-514 view + export; Member 1 validates WP-514 field mapping against mentor feedback
Day 2, Hr 6–12	Run all 9 seeded test cases end-to-end; fix mismatches; polish dashboard styling and evidence viewer

When	Milestone
Day 2, Hr 12–16	Bug-bash + edge cases (malformed PDF, OCR fallback, missing note); tighten AI prompts so explanations stay grounded and clearly labelled
Day 2, Hr 16–20	Freeze features; write the demo script; rehearse the GreenPeak-style walkthrough end-to-end at least twice
Day 2, Hr 20–24	Final rehearsal, record a backup demo video, prepare the submission write-up (problem, solution, architecture, impact) and submit

10. Aligning the Build to Judging Criteria

Cognizant's hackathon format typically scores submissions on Impact, Innovation, Technical Strength and Scalability. Map the demo narrative accordingly:

Criterion	How This MVP Delivers It
Impact	Directly cuts manual review hours for a real, recurring banking back-office task; every finding is evidence-linked, so it is usable, not a black box
Innovation	Deliberate deterministic/AI split — LLM is grounded and never trusted with arithmetic — a defensible pattern for regulated financial workflows
Technical Strength	Real extraction from PDF/Excel, tested rule engines, structured LLM output, evidence tracing, working end-to-end dashboard demo
Scalability	Rule library is explicitly extensible; canonical JSON layer decouples extraction from checks so new statement formats or banks plug in without rewriting engines; stateless services are container-ready for horizontal scaling

11. What NOT to Overbuild

- Do not implement every possible accounting rule — ship a strong, tested core (the 5 checks in §6.3/6.4) and keep the rule library extensible.
- Do not use an LLM for any deterministic calculation — keep that boundary absolute, it is the credibility of the whole product.
- Do not spend hackathon hours on authentication, microservices, or production infra — one FastAPI process + Docker Compose is enough to demo.
- Do not build a large dataset — the 9 seeded cases from the original plan (§5 of the source document) are sufficient to prove every module.
- Do not let the AI layer's explanations look like established fact — always label them as suggested/pending reviewer confirmation.

12. Post-Hackathon Scalability Roadmap

- Swap synchronous checks for an async task queue (Celery/RQ or a lightweight job runner) as document volume grows.
- Add a rules-config layer (YAML/JSON rule definitions) so new checks don't require code changes.
- Introduce role-based auth and audit-trail logging once moving beyond demo.
- Add a vector-indexed note/disclosure store so the AI layer can ground explanations across many notes and prior filings, not just the current document.
- Support additional input formats (XBRL, scanned/OCR-heavy filings) by extending the extraction layer only — canonical JSON and downstream engines stay untouched.

13. Integration Rulebook — Rules Every Member Follows

Most hackathon integrations break not because the code is bad, but because two members quietly assumed different shapes for the same data. With 7 people building in parallel, the fix is to freeze a small number of contracts on Day 1, Hour 0–2, and never let anyone deviate from them without a team-wide change. The rules below are what each member commits to.

13.1 Repository & Branching

- One monorepo, not 7 separate repos — folders: /frontend, /backend/extraction, /backend/math_engine, /backend/consistency_engine, /backend/ai_layer, /backend/wp514, /backend/orchestrator, /data/synthetic.
- Branch per member per feature: feature/<member>-<short-task>, e.g. feature/m4-math-engine. Never commit directly to main.
- Every merge to main goes through a pull request with at least one other member's review — even at 2am. A 2-minute review catches a schema mismatch before it costs an hour of debugging.
- main must always run end-to-end on the sample fixture case, even if only 1 of 9 test cases passes fully — never merge something that breaks the pipeline for everyone else.

13.2 The One Contract Everyone Must Honor: Canonical JSON

The canonical financial data schema (§6.2) is versioned and frozen at kickoff (schema_version: "1.0"). It is the single hand-off point between extraction (Member 3) and every engine (Members 4, 5, 6). Rules:

- Nobody changes a field name, nesting level, or unit convention in the canonical schema solo — changes are proposed in the team chat and agreed by whoever consumes that field before merging.
- If a field is genuinely missing for a case (e.g. no Note 12 exists), it is set to null with a "not_found" evidence reason — never omitted silently. Downstream engines must treat null as "cannot check," not "zero."
- Member 3 (extraction) publishes 2–3 example canonical JSON fixture files by Hour 6 on Day 1 so Members 4, 5 and 6 can start building against real-shaped data without waiting on the extractor being finished.

13.3 Findings Contract (Math, Consistency, AI must all emit this shape)

Every check — regardless of which member's module produced it — returns the same Finding object (see §8 for the full JSON shape): finding_id, module, check, status (pass/exception/not_applicable), severity, expected, actual, difference, evidence[], ai_explanation. The Findings Aggregator and the Dashboard only ever read this one shape, so any engine can be developed, tested and swapped independently.

- status must be one of exactly three values: "pass", "exception", "not_applicable" — no free-text status strings, or the dashboard's filter/severity logic breaks.
- severity is only ever "high", "medium" or "low", assigned by the rule itself (e.g. balance sheet mismatch = high; a minor rounding diff = low).
- evidence[] is never empty for a pass or an exception — a check that can't point to its source data isn't trustworthy enough to show a reviewer.

13.4 Evidence Pointer Standard

Every number that flows through the system — from extraction onward — carries the same evidence shape, so the dashboard's drill-down works for every module without special cases:

```
{ "doc_id": "greenpeak_fy25.pdf", "page": 2, "table": "Balance Sheet", "row": "Total Assets",
  "period": "FY2025" }
```

- Member 3 attaches this to every extracted value at source.
- Members 4, 5 and 6 only ever copy evidence pointers forward into their findings — they never invent or edit one.

13.5 Naming, Units & Formatting Conventions

Convention	Rule
Field names	snake_case everywhere — JSON, Python and API alike (total_assets, not TotalAssets or Total Assets)
Currency unit	One unit for the whole pipeline, agreed at kickoff (e.g. ₹ Crore, 2 decimal places) — extraction converts to it immediately so no engine ever guesses scale
Periods	Always "FY2025", "FY2024" style strings, never raw dates, so every module can join on the same key
Percentages	Stored as plain floats (0.34 for 34%), formatted only at the UI layer
IDs	job_id and finding_id are UUIDs generated once by the orchestrator and passed through unchanged by every module

13.6 API & Interface Rules

- Only the frontend talks to the backend over HTTP. Internally, extraction → math/consistency → AI → WP-514 are Python function calls inside one orchestrator pipeline, not separate network services — this removes an entire class of “my service is down” integration bugs during the hackathon (see §14 for why).
- Every backend function that a teammate depends on gets a one-line docstring stating its exact input and output shape before anyone else builds against it.
- Breaking a function's signature requires a heads-up in the team channel — “I'm changing math_engine.run() to also take a materiality_threshold param” — before pushing, not after.

13.7 Environment & Config

- One .env.example checked into the repo covering: DATABASE_URL, CLAUDE_API_KEY, STORAGE_PATH, MATERIALITY_THRESHOLD_PCT. Every member copies it to their own .env — nobody hardcodes secrets or paths.
- One docker-compose.yml starts the whole stack (db + backend + frontend) with a single command — this is what gets demoed to judges, so it must be tested by someone other than its author before Day 2 evening.

13.8 Testing Requirement Before Any Merge

- Members 4, 5, 6: every rule/check function ships with at least one unit test against a fixture canonical JSON (a clean case and a broken case) — verified before merging, not after.
- Member 3: extraction is verified against all of Member 2's synthetic files, with a checked-in “expected canonical JSON” per file to diff against.
- Member 7: one integration test that runs a full fixture case through the entire pipeline (extraction → findings → WP-514) and asserts the known seeded errors appear — this is the test that protects the demo.

13.9 Definition of Done (per member, before calling a task finished)

Member	Not Done Until...
1 — Domain/WP-514	Every WP-514 field is mapped to a specific finding field, confirmed against mentor feedback

Member	Not Done Until...
2 — Dataset	Ground truth JSON exists per case listing every seeded error and its expected finding
3 — Extraction	Canonical JSON output matches the frozen schema exactly, verified on all synthetic cases
4 — Math Engine	Every rule in §6.3 has a passing unit test on a clean case and a failing case
5 — Consistency + PY	Every rule in §6.4 runs against real extracted output, not just fixtures
6 — AI Layer	Every AI output is valid JSON matching the Finding shape and never contains a number absent from its input
7 — Dashboard	A judge can upload a file and see status → findings → evidence → WP-514 without any manual step

13.10 Integration Checkpoints (tie-in to the Day 1/Day 2 timeline)

- End of Day 1, Hr 18–24: mandatory full-team integration checkpoint — run one real case end-to-end together, on one screen, and fix whatever breaks before anyone starts new features.
- Day 2, Hr 6–12: second checkpoint — run all 9 seeded cases; any case that fails gets a named owner on the spot.
- Day 2, Hr 16: feature freeze — from this point only bug fixes are allowed, no new checks or UI features, so the demo stays stable.

14. Detailed Internal System Architecture (For Reviewer Walkthroughs)

This section is written so any member can stand in front of a judge and explain exactly what happens, step by step, from the moment a file is dropped in to the WP-514 output appearing on screen — including what the system does when something goes wrong.

14.1 Architectural Style: Modular Monolith, Not Microservices

Deliberately not a microservices system. One FastAPI backend process contains clearly separated Python packages (extraction, math_engine, consistency_engine, ai_layer, wp514, findings), called in sequence by a single orchestrator function. This is a defensible, explainable technical choice for a 2-day build:

- No network calls between modules → no partial-failure states between them, no service discovery, nothing extra to containerize or debug under time pressure.
- Each package still has a single clear responsibility and a typed interface, so it could be split into a real microservice later without a rewrite — the roadmap in §12 covers exactly that path.
- This is the answer to give a judge who asks “why isn't this microservices”: it was a deliberate scope decision, not a limitation — the module boundaries already exist in code, they're just not yet network boundaries.

14.2 Component Responsibility Table

Component	Input	Output	Type
Frontend (React)	User actions (upload, click)	HTTP calls to backend API	Browser SPA
Orchestrator (FastAPI)	Uploaded file + job_id	Job status, triggers pipeline stages in order	API + pipeline controller
Extraction package	Raw PDF/XLSX bytes	Canonical JSON + evidence pointers	Python function, pure(ish) — I/O only for OCR/file read

Component	Input	Output	Type
Math Engine package	Canonical JSON	List of Finding objects (deterministic)	Pure Python functions
Consistency + PY package	Canonical JSON	List of Finding objects (deterministic)	Pure Python functions
AI Layer package	Canonical JSON (validated) + flagged findings	ai_explanation fields + summary text	Claude API call, grounded prompt
Findings Aggregator	All Finding lists	Merged, deduplicated findings + overall status	Pure Python function
WP-514 package	Merged findings	WP-514 draft record	Pure Python function (template mapping)
Database (Postgres)	All of the above	Persisted jobs, canonical data, findings, WP-514	Relational store

14.3 End-to-End Request Lifecycle (What Actually Happens, Step by Step)

#	Step	What Happens Internally
1	Upload	POST /api/jobs (multipart file) → orchestrator saves raw file to /storage/{job_id}/raw.pdf, inserts a row into jobs (status = UPLOADED), returns job_id to the frontend immediately
2	Pipeline start	Orchestrator calls extraction.run(job_id) as a background task; job status → EXTRACTING
3	Extraction	pdfplumber/camelot/openpyxl parse the file; output is normalized into canonical JSON with evidence pointers; saved to canonical_data table; status → EXTRACTED (or EXTRACTION_FAILED on error, see §14.6)
4	Deterministic checks	Orchestrator calls math_engine.run() and consistency_engine.run() with the canonical JSON (can run concurrently — neither depends on the other); each returns a list of Finding objects; status → CHECKED
5	AI layer	Orchestrator calls ai_layer.run(canonical_data, findings), passing only validated figures and any exception/flagged findings; returns grounded explanations, grammar suggestions and a summary; merged into findings; status → EXPLAINED (AI failure does not block the pipeline, see §14.6)
6	Aggregation	Findings Aggregator merges all three sources into one list, computes overall status (Pass / N Exceptions) and severity counts; saved to findings table
7	WP-514 generation	wp514.generate(findings) maps validated findings into the WP-514 template fields; saved to wp514_entries; status → WP514_READY, then DONE
8	Dashboard polling	Frontend polls GET /api/jobs/{id} until status = DONE, then fetches GET /api/jobs/{id}/findings and GET /api/jobs/{id}/wp514 to render the results
9	Evidence drill-down	Reviewer clicks a finding → frontend calls GET /api/evidence/{evidence_id} → backend returns the source page image with the relevant row/cell highlighted
10	Sign-off	Reviewer edits/approves a WP-514 row → PATCH /api/wp514/{id} — the only write path a human controls directly, keeping sign-off human-controlled per §6.6

14.4 Core Database Schema

```

jobs          (id, bank_name, filename, status, created_at, updated_at)
canonical_data (job_id FK, schema_version, data JSONB)
findings      (id, job_id FK, module, check, status, severity,
              expected, actual, difference, evidence JSONB,
              ai_explanation, created_at)
wp514_entries (id, job_id FK, item, expected, actual, status,
```

```
evidence_ref, reviewer_comment, signed_off_by)
evidence_assets (id, job_id FK, doc_id, page, image_path)
```

14.5 Key API Endpoints

Endpoint	Purpose
POST /api/jobs	Upload a statement file, create a job, kick off the pipeline
GET /api/jobs/{id}	Poll job status (UPLOADED → EXTRACTING → CHECKED → EXPLAINED → DONE)
GET /api/jobs/{id}/canonical	Debug view of extracted canonical data (used during development, not judge-facing)
GET /api/jobs/{id}/findings	List of all findings for the dashboard's findings table
GET /api/jobs/{id}/wp514	WP-514 draft for the WP-514 tab
PATCH /api/wp514/{id}	Reviewer edits a WP-514 row or signs it off
GET /api/evidence/{id}	Returns the source page image/crop for a given evidence pointer

14.6 Failure Modes — What the System Does When Something Breaks

Reviewers often probe robustness. Having explicit, designed-for failure behaviour (rather than “it would crash”) is a strong technical-strength signal.

Failure	System Behaviour
Corrupt/unreadable PDF	Job status → EXTRACTION_FAILED with a human-readable reason; dashboard shows an actionable error instead of a blank screen; pipeline does not proceed to the engines
Scanned page, no text layer	Falls back to Tesseract OCR for that page; if OCR confidence is low, the affected fields are marked null with reason "ocr_low_confidence" rather than a guessed number
A note/statement genuinely missing	Consistency checks that depend on it return status = not_applicable, not a false exception — the dashboard shows it as “not checked”, not “failed”
AI API timeout or error	Deterministic findings (math + consistency) still populate and display normally; affected findings show ai_explanation: null with an “explanation unavailable” badge; the pipeline still reaches WP-514 and DONE
Two members' schema drift	Caught by Member 7's integration test (§13.8) before merge — CI fails loudly rather than surfacing as a silent bug during the live demo

14.7 60-Second Narration Script for Judges

A ready-to-use spoken walkthrough that maps directly onto the architecture, for whoever demos:

- “We upload a bank's financial statements — the system extracts every figure into a normalized structure, keeping a pointer back to the exact page and row it came from.”
- “Two deterministic engines then cross-check everything a calculator could check — totals, cross-casting, prior-year movements — with zero AI involved, so those results are fully reproducible.”
- “Only after the numbers are validated does an AI layer step in — and only to explain and phrase, grounded strictly in those validated figures, never to invent a number.”
- “Every result — pass or exception — is evidence-linked, so a reviewer can verify it in one click, and validated findings auto-populate a WP-514 working paper while keeping sign-off in human hands.”

Appendix A — Seeded Test Case Matrix

Case	Scenario	Expected Result
1	Clean statements	Core checks pass
2	Balance Sheet math error	Math engine identifies mismatch and amount
3	P&L / Cash Flow calculation error	Relevant check fails
4	Prior-year mismatch	PY difference identified
5	Statement vs note mismatch	Consistency issue flagged with evidence
6	Large unusual movement	Analytics flags it; AI explains it
7	Spelling/grammar issues	Language engine suggests corrections
8	Missing/inconsistent disclosure	Disclosure/WP-514 review flags it where supported
9	Multiple issues in one bank	Dashboard consolidates all findings (see §7 walkthrough)

Appendix B — One-Line Demo Goal

Upload a fictional bank's financial statements → automatically detect planted review issues → show source evidence → explain the issues → populate the applicable WP-514 output — all in a single, judge-facing, end-to-end run.